



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 11

“Memoria EEPROM”

Grupo: 6CM1

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 27 de febrero de 2025

INTRODUCCIÓN

Memoria EEPROM

La memoria EEPROM (Electrically Erasable Programmable Read-Only Memory) representa una evolución crítica en la jerarquía de memorias de estado sólido, situándose entre la rigidez de la ROM y la volatilidad de la RAM.

Es un tipo de memoria no volátil, lo que significa que conserva la información almacenada incluso cuando se retira la fuente de alimentación. A diferencia de sus predecesoras (como la EPROM, que requería luz ultravioleta para borrarse), la EEPROM permite el borrado y la reprogramación mediante señales eléctricas a nivel de byte.

El transistor de puerta flotante (Floating gate)

El componente fundamental de una celda EEPROM es un transistor MOS modificado. Este posee una "puerta flotante" aislada por una capa de óxido de alta calidad.

- **Almacenamiento:** Los electrones son atrapados en esta puerta mediante un fenómeno físico llamado Efecto Túnel (Fowler-Nordheim Tunneling).
- **Estados:** La presencia o ausencia de carga en esta puerta modifica el umbral de conducción del transistor, representando así un 1 o un 0 lógico.

Características operacionales

Para entender su lugar en la electrónica, es necesario analizar sus parámetros de rendimiento:

- **Acceso a Nivel de Byte:** A diferencia de la memoria Flash (que borra y escribe en bloques grandes), la EEPROM permite modificar una dirección de memoria específica sin afectar a las demás. Esto la hace ideal para almacenar parámetros de configuración pequeños y frecuentes.
- **Resistencia (Endurance):** Las celdas EEPROM tienen una vida útil finita. El proceso de tunelización degrada gradualmente la capa de óxido. Típicamente, soportan entre **100,000 y 1,000,000 de ciclos** de escritura/borrado.
- **Retención de Datos:** Se refiere al tiempo que la carga permanece atrapada en la puerta flotante sin degradarse. Los estándares industriales suelen garantizar más de 10 o 20 años de retención.
- **Latencia de Escritura:** Escribir en una EEPROM es significativamente más lento que escribir en una RAM debido al tiempo requerido para estabilizar las cargas eléctricas en la celda.

Algunas aplicaciones

la EEPROM se reserva para datos que cumplen tres condiciones: son vitales, son pequeños y cambian ocasionalmente.

- **Calibración:** Almacenamiento de valores de compensación de sensores.
- **Identificación:** Números de serie únicos y direcciones MAC.
- **Estado de Usuario:** Último canal sintonizado en una TV o preferencias de usuario en un sistema embebido.

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 11, el circuito a armar es el siguiente:

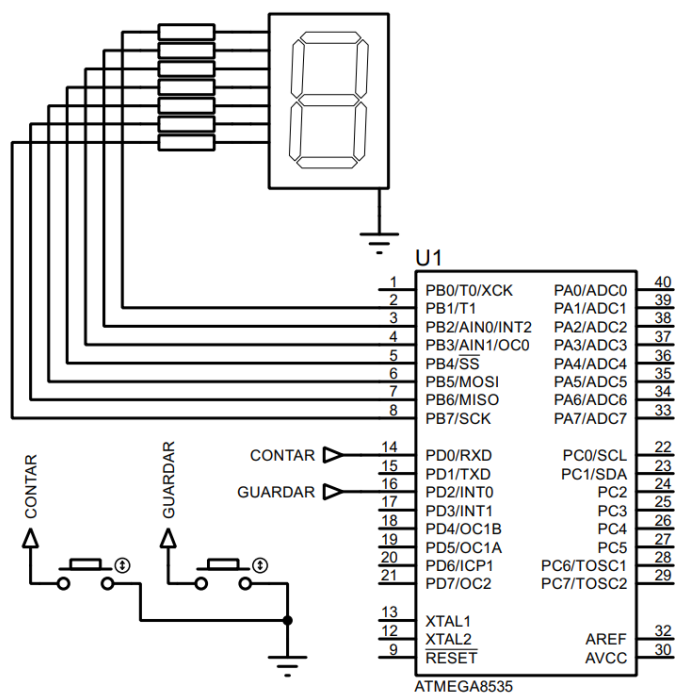


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen del circuito, el registro B se configura como de salida (PB1 – PB7), mientras que el puerto D como de entrada (PD0 y PD2), activando las resistencias de pull-up.

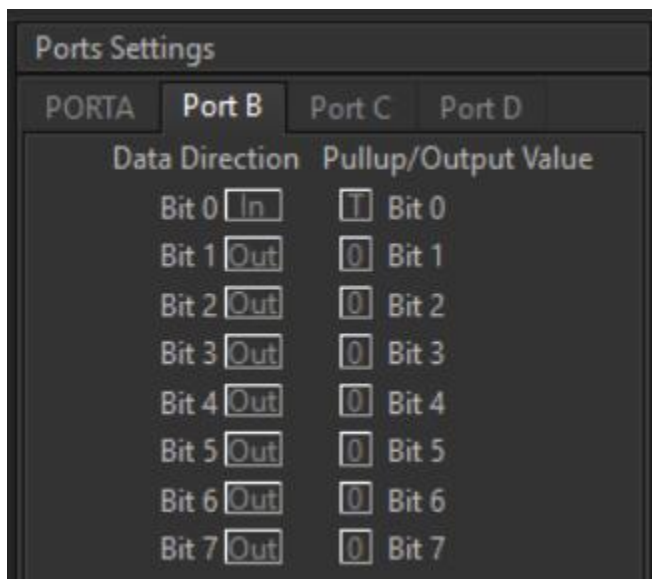


Imagen 2. Configuración del puerto B.

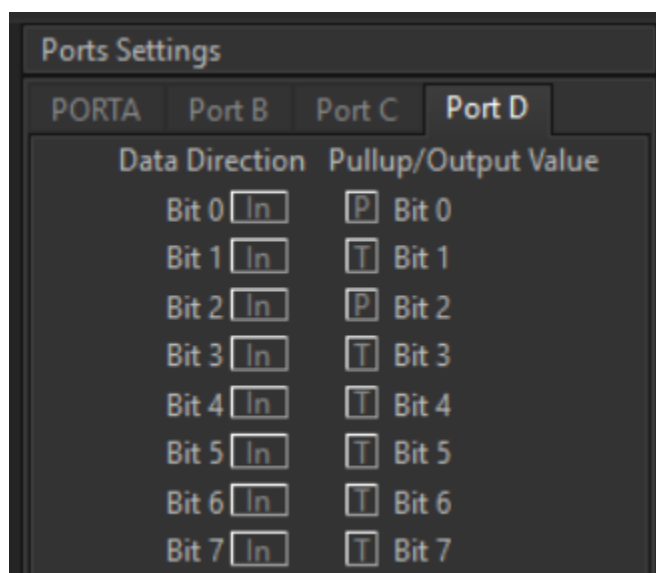


Imagen 3. Configuración del puerto D.

Para poder hacer el contador es necesario interpretar el número correspondiente del display como un número hexadecimal, para ello se realizó la siguiente tabla, tomando en cuenta lo que se indicó en la práctica 3:

Número Display	Combinaciones								Valor hexadecimal
	g	f	e	d	c	b	a		
0	0	1	1	1	1	1	1	0	0x7e
1	0	0	0	0	1	1	0	0	0x0c
2	1	0	1	1	0	1	1	0	0xb6
3	1	0	0	1	1	1	1	0	0x9e
4	1	1	0	0	1	1	0	0	0xcc

5	1	1	0	1	1	0	1	0	0xda
6	1	1	1	1	1	0	1	0	0xfa
7	0	0	0	0	1	1	1	0	0x0e
8	1	1	1	1	1	1	1	0	0xfe
9	1	1	0	1	1	1	1	0	0xde
A	1	1	1	0	1	1	1	0	0xee
B	1	1	1	1	1	0	0	0	0xf8
C	0	1	1	1	0	0	1	0	0x72
D	1	0	1	1	1	1	0	0	0xbc
E	1	1	1	1	0	0	1	0	0xf2
F	1	1	1	0	0	0	1	0	0xe2

Tabla 1. Representaci3n hexadecimal para los n3meros del 0 al 9 y del A a la F en hexadecimal.

C3digo del circuito de CodeVision:

```

1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4  Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Pr3ctica 11 'Memoria EEPROM'
8  Version : 1.0
9  Date    : 25/02/2026
10 Author  : Garc3a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type           : ATmega8535
16 Program type        : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model         : Small
19 External RAM size     : 0
20 Data Stack size      : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25 #include <delay.h>
26 #define BOTON_CONTAR PIND.0
27 #define BOTON_GUARDAR PIND.2

```

```

29 // Declare your global variables here
30 #include <avr/io.h>
31
32 void main(void) {
33     // Declare your local variables here
34     const char hex[16] = {0x7e, 0x0c, 0xb6, 0x9e, 0xcc, 0xda, 0xfa, 0xe, 0xfe, 0xde, 0xee, 0xf8, 0x72, 0xbc, 0xf2, 0xe2};
35     bit incremento_pasado, incremento_actual, guardado_pasado, guardado_actual;
36     unsigned char indice;
37
38     // Input/Output Ports initialization
39     // Port A initialization
40     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
41     DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
42     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
43     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
44
45     // Port B initialization
46     // Function: Bit7=Out Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=In
47     DDRB=(1<<ddb7) | (1<<ddb6) | (1<<ddb5) | (1<<ddb4) | (1<<ddb3) | (1<<ddb2) | (1<<ddb1) | (1<<ddb0);
48     // State: Bit7=0 Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=T
49     PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
50
51     // Port C initialization
52     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
53     DDRC=(0<<DDC7) | (0<<DDC6) | (0<<DDC5) | (0<<DDC4) | (0<<DDC3) | (0<<DDC2) | (0<<DDC1) | (0<<DDC0);
54     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
55     PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);

```

```

57     // Port D initialization
58     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
59     DDRD=(0<<DDD7) | (0<<DDD6) | (0<<DDD5) | (0<<DDD4) | (0<<DDD3) | (0<<DDD2) | (0<<DDD1) | (0<<DDD0);
60     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=P Bit1=T Bit0=P
61     PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (1<<PORTD2) | (0<<PORTD1) | (1<<PORTD0);
62
63     // Timer/Counter 0 initialization
64     // Clock source: System Clock
65     // Clock value: Timer 0 Stopped
66     // Mode: Normal top=0xFF
67     // OC0 output: Disconnected
68     TCCR0=(0<<WGM00) | (0<<COM01) | (0<<COM00) | (0<<WGM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
69     TCNT0=0x00;
70     OCR0=0x00;

```

```

72     // Timer/Counter 1 initialization
73     // Clock source: System Clock
74     // Clock value: Timer1 Stopped
75     // Mode: Normal top=0xFFFF
76     // OC1A output: Disconnected
77     // OC1B output: Disconnected
78     // Noise Canceler: Off
79     // Input Capture on Falling Edge
80     // Timer1 Overflow Interrupt: Off
81     // Input Capture Interrupt: Off
82     // Compare A Match Interrupt: Off
83     // Compare B Match Interrupt: Off
84     TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
85     TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
86     TCNT1H=0x00;
87     TCNT1L=0x00;
88     ICR1H=0x00;
89     ICR1L=0x00;
90     OCR1AH=0x00;
91     OCR1AL=0x00;
92     OCR1BH=0x00;
93     OCR1BL=0x00;

```



```

95 // Timer/Counter 2 initialization
96 // Clock source: System Clock
97 // Clock value: Timer2 Stopped
98 // Mode: Normal top=0xFF
99 // OC2 output: Disconnected
100 ASSR=0<<AS2;
101 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
102 TCNT2=0x00;
103 OCR2=0x00;
104
105 // Timer(s)/Counter(s) Interrupt(s) initialization
106 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
107
108 // External Interrupt(s) initialization
109 // INT0: Off
110 // INT1: Off
111 // INT2: Off
112 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
113 MCUCSR=(0<<ISC2);
114
115 // USART initialization
116 // USART disabled
117 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);

```

```

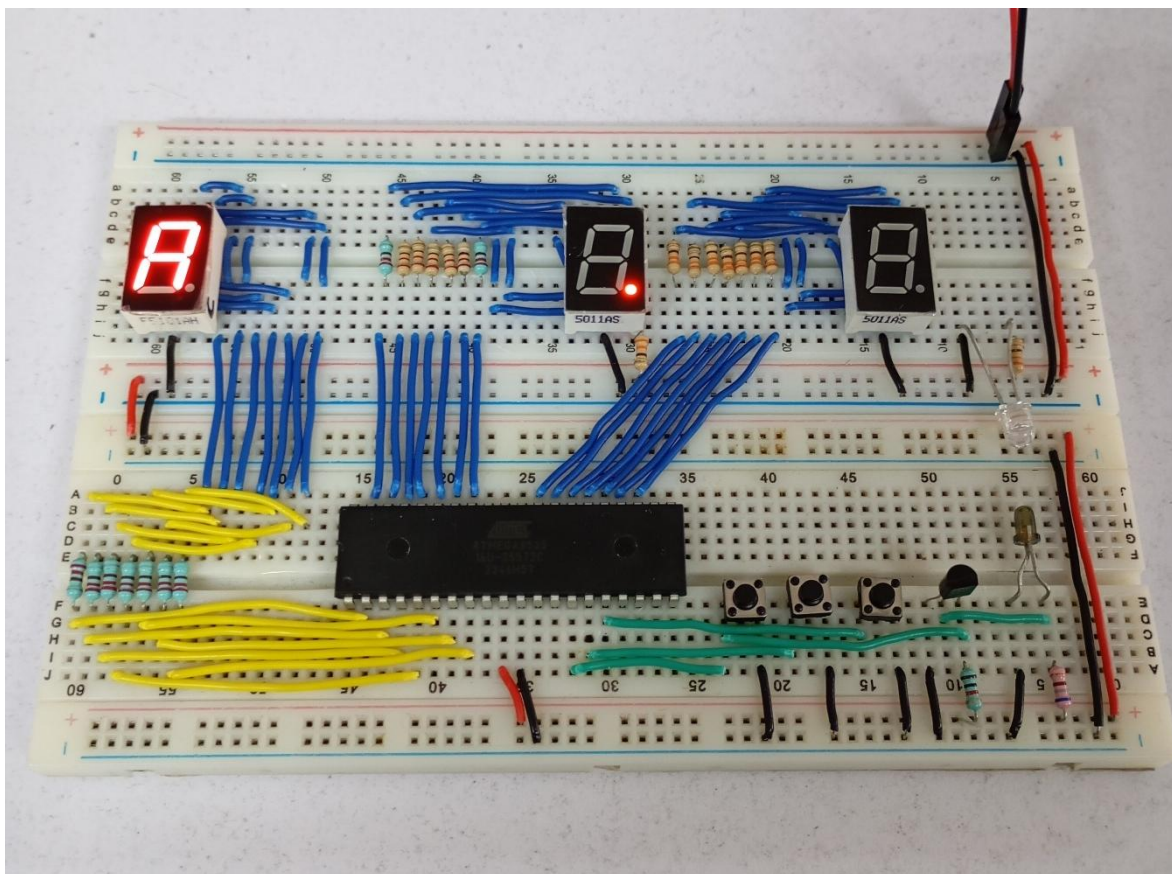
119 // Analog Comparator initialization
120 // Analog Comparator: Off
121 // The Analog Comparator's positive input is
122 // connected to the AIN0 pin
123 // The Analog Comparator's negative input is
124 // connected to the AIN1 pin
125 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
126 SFIOR=(0<<ACME);
127
128 // ADC initialization
129 // ADC disabled
130 ADCSRA=(0<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (0<<ADPS0);
131
132 // SPI initialization
133 // SPI disabled
134 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
135
136 // TWI initialization
137 // TWI disabled
138 TWCR=(0<<TWEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);
139
140 // Se carga el último valor guardado en la memoria EEPROM
141 indice = guardable;

```

```
143 while (1) {
144     // Place your code here
145     incremento_actual = BOTON_CONTAR;
146     guardado_actual = BOTON_GUARDAR;
147
148     // El bot3n de contar se presiona
149     if (incremento_pasado && !incremento_actual) {
150         indice++;
151         if (indice > 15) indice = 0;
152         delay_ms(40);
153     }
154
155     // El bot3n de contar se suelta
156     if (!incremento_pasado && incremento_actual) {
157         delay_ms(40);
158     }
159
160     // El bot3n de guardar se presiona
161     if (guardado_pasado && !guardado_actual) {
162         // Guardar el dato actual en la memoria EEPROM
163         guardable = indice;
164         delay_ms(40);
165     }
```

```
167     // El bot3n de guardado se suelta
168     if (!guardado_pasado && guardado_actual) {
169         delay_ms(40);
170     }
171
172     // Actualizar estados de los botones
173     incremento_pasado = incremento_actual;
174     guardado_pasado = guardado_actual;
175
176     // Mostrar el dato en el display
177     PORTB = hex[indice];
178 }
179 }
```


Circuito físico



CONCLUSIÓN

- **García Escamilla Bryan Alexis**

En esta práctica se aprendió que los datos no solo se pueden almacenar en la memoria RAM, sino que también en la memoria EEPROM, que a diferencia de la RAM esta conserva la información almacenada incluso cuando se retira la fuente de alimentación.

Para usar la memoria EEPROM basta con anteponer la palabra **EEPROM** en la definición de una variable que posteriormente se usara para asignarle un valor, que es que guardara en la memoria.

Para el caso específico del ATMEGA8535 el límite de escrituras es de 100,000, es por eso que es importante controlar cada cuando se escribe en la memoria EEPROM, para ello se usó la detección de flancos en los botones para evitar rebotes y así escrituras no deseadas.

- **Meléndez Macedonio Rodrigo**

Para esta práctica se trabajó con un concepto muy interesante el cual es la Memoria EEPROM, que, a diferencia de la Memoria RAM, esta tiene la cualidad de que puede guardar los datos aun cuando el circuito deja de recibir alimentación. Para ello, CodeVisionAVR

ofrece ciertas facilidades con este tipo de memoria pues solo basta con escribir en el código la palabra "**ee**pr**om**" al momento de definir las variables y luego asignarle un valor específico con el que será capaz de guardar la información que nosotros le indiquemos.

En este caso, para verificar que la información se queda guardada en el ATMEGA8535 usamos un display de 7 segmentos en el que guardamos valores del 0 a F mediante un botón específico que se encarga de guardar el dato que nosotros le indiquemos, logrando el objetivo de la práctica.

BIBLIOGRAFÍA

(s.f.). *EEPROM*. Wikipedia. Recuperado el 7 de marzo de 2026.
<https://es.wikipedia.org/wiki/EEPROM>

(s.f.). *EEPROM Definition*. TechTarget. Recuperado el 7 de marzo de 2026.
<https://www.techtarget.com/>